TD HDLC

Sujet (https://moodle-n7.inp-toulouse.fr/mod/resource/view.php?id=83506)

Part1

1.1

hdlc -> couche 2
mettre un flag/fanion pour délimiter 2 trame (01111110 indique le début d'une trame et sa fin -> en cas de 2 trames d'affilé, on ne met que le fanion de début (01111110 hdlc1 01111110 hdlc2 etc)

(on rajoute un 0 tout les 5 1 pour éviter de trouver le flag dans les données)

flag sert aussi à syncro l'horloge sur le temps bit

envoie d'un octet représentant un chr vide [SYNC] pour faire la synchro, envoie d'un chr [STX] avant la data et [ETX] à la fin (cas de donnée texte sous forme ascii)
-> pb: [ETX] n'arrive qu'après un nombre entier d'octet (pas de données binaire)
plus simple de resynchro en cas d'erreur quand lecture bit à bit (avec des flag) qu'avec des octets/ chr particulier

1.2

trame étudié : [10110110 11111010 10111111 10010111]
message effectivement envoyé : ajoute fanion en strat et end + add (0) tout les 5 1 d'affilé
-> *(01111110)* 101**1**0110 11111(**0**)010 1011111(0)1 10010111 *(0111111**1**0)*
à la reception : on enlève les flag de début et de fin + on retire les 0 si on lit 5 1 à la suite

si 12eme bit faux => adresse éronnée
si 22eme bit faux => apparition d'un flag -> perte de la trame entière
si 48eme bit faux => perte du fanion de fin -> perte la trame et éventuellement de la suivante (non détéction de la fin de la trame)

Part 2

1.1

cas taux d'erreur nul (couche 2) -> tout va bien
cas taux d'erreur non nul (couche 2) ->

2.2

> full duplex -> les 2 sens en même temps
> half duplex -> un seul sens à la fois

NRM -> mode normal de réponse hdlc -> champs adresse pour dire à qui parle la station principale, un bit PF qui annonce si la Trx est fini (station secondaire ne peuvent pas émettre avant d'y être autorisé par la station principale)
ABM -> mode asynchrone équilibré (en point à point) -> connexions entre les stations qui peuvent émettre quand elles le souhaitent
A/ **ABM** sur half duplex possible ? non besoin de full duplex
B/ **NRM** sur full duplex possible ? oui (pas grand intérêt sauf si plusieurs émetteur en même temps)

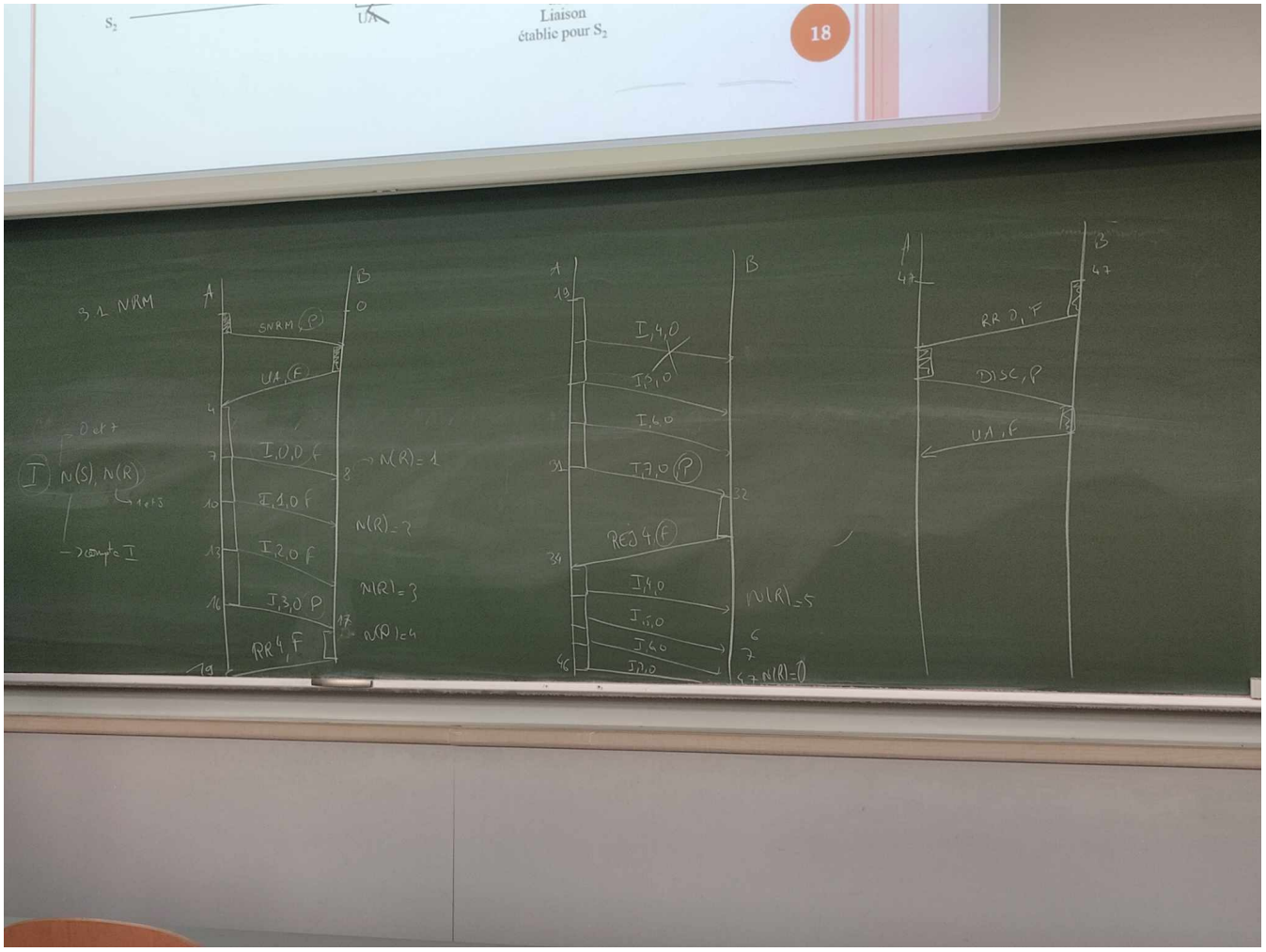
2.3

permet le contrôle de flux en plus (comparé à ethernet et LLC)

Part 3

3.1

(NRM)
(P) -> bit p à 1 -> B à le droit d'émettre
(F) -> B à terminer d'envoyer sa réponse
I N(s)N® -> N(s) nbr de trame courrente (entre 0 et 7) N® nbr de trame que j'ai reçu (entre 1 et 8)



tps:

(0) A envoie SNRM(p) à B

(2) B envoi UA(F) à A

(4) A envoi I00(F) B reçois à (8)

(7) A envoi I10(F) B reçois à (11)

(10) A envoi 120(F) B reçois à (14)

(13) A envoi I30(P) B reçois à (17)

(18) B envoi RR4(F) A reçois à (19)

(19) A envoi I40 (trame perdu)

(22) A envoi I50

(25) A envoi I60

(28) A envoi I70 B reçois à (32)

(33) B envoi REJ4(F)

(34) A renvoi I40

(37) A renvoi I50

(40) A renvoi I60

(43) A renvoi I70

(47) B envoi RR0(F)

() A envoi DISC(P)

() B envoi UA(F)

3.2

(ABM)
pas besoin du bit PF
Ixy :

> x : num de message envoyé (entre 0 et 7)
> y : nbr de message reçu (entre 0 et 7) -> numéro de la prochaine trame que tu peux m'envoyer
> cc : l'acquitement d'une trame se "cache" dans la data, pas besoin de trame d'acquitement

quand A reçois I34, detecte qu perte du 2eme et 3eme msg de B et envois un SREJi avec i = le numéro du msg à renvoyer (donc un SREJ1 puis un SREJ2 à la reception du I14 de B)

